AWS Trainium2 (trn2.48xlarge) 评测报告

日期: 2026-11-07 · **区域**: us-east-2b (Capacity Block, 29h, $1037.06) **实例**: i-0xxxxxxxxxxxxxxxx · **AMI**: Neuron DLAMI Ubuntu 24.04, SDK 2.33.0 **软件栈**: torch-neuronx 2.14（原生 `neuron` device）、neuronx-cc 2.28、NRT 2.35、vLLM-Neuron 0.29、nccom-test

0. 硬件拓扑

- 16× Trainium2 芯片，LNC=2 逻辑核配置 → **64 个逻辑 NeuronCore**，每个 24 GB HBM（全机 1.5 TB HBM）
- 每芯片与 4 个邻居通过 NeuronLink 直连（2D torus），跨 2 个 NUMA 域
- 192 vCPU、2 TB 内存、4×1.9 TB NVMe（本测试组 RAID0 = 7 TB）
- 本实例未附加 EFA 网卡（单机评测；vLLM 需 `NEURON_SKIP_EFA_AFFINITY=1`）

> 下文 "per-core" 均指 LNC=2 逻辑核 = 半颗芯片的一半（即 1/4 芯片的 2 个物理核）； 1 芯片 = 4 逻辑核。
> 换算：aggregate(chip) = 4 × per-core。

1. GEMM / PFLOPs

方法：`y = y @ W` 链式方阵乘（16 连乘成图），BF16/FP32，每逻辑核独立执行（数据并行微基准，无通信）。

dtype	ranks	N	per-core TFLOPs	聚合 TFLOPs
bf16	1	4096	140.4	140
bf16	1	8192	**148.6**	149
bf16	1	16384	145.5	145
bf16	4 (1 chip)	4096	118.7	**475**
bf16	4 (1 chip)	8192	106.5	426
bf16	64 (整机)	4096	118.6	**7591**
bf16	64 (整机)	8192	105.2	6730
fp32	1	4096	36.3	36

结论

- 单逻辑核 BF16 峰值 **148.6 TF** ≈ 单芯片 594 TF，达标称密集 BF16 (~667 TF/chip) 的 **89%**。
- 整机聚合 **7.59 PFLOPs BF16**（@N=4096），为标称 ~10.7 PF 的 **71%**；全核并发时 per-core 从 148→118 TF， 主因 HBM 带宽/功耗在芯片内 4 核间共享。
- FP32 ≈ BF16 的 1/4（36 TF/core），符合架构比例。N=1024 时矩阵太小吃不满脉动阵列（43 TF）。

2. HBM 带宽

两条路径：

a) DMA `copy_`（真实 HBM 带宽，读+写）

ranks	数组	per-core GB/s	整机聚合
1	1 GB	640	—
64	1 GB	643	**41.2 TB/s**

单核与 64 核并发 per-core 带宽一致（640 ≈ 643 GB/s）→ 每逻辑核的 DMA 通路独立， 单核即可打满自己那份 HBM 切片，无跨核争抢。

整机 41.2 TB/s ≈ 标称 46.4 TB/s (2.9 TB/s × 16) 的 **89%** —— HBM 实测非常接近规格。

b) STREAM copy/scale/add/triad（框架 elementwise 路径，fp32，1 GB/数组）

kernel	per-core GB/s	整机聚合 GB/s
copy	80	5 103
scale	160	10 216
add	147	9 437
triad	91	5 847

框架 elementwise 只有 DMA 路径的 ~25%：瓶颈在计算引擎的逐元素吞吐而非 HBM （旁证：1→4→64 核完美线性扩展，无带宽争抢）。在 Trainium 上带宽敏感算子应依赖 编译器融合/DMA，手写 elementwise 链不划算。

3. H2D / D2H（PCIe）

ranks	块大小	H2D（单流/聚合）	D2H（单流/聚合）
1	4 GB	12.9 GB/s	3.4 GB/s
16	1 GB	2.0 / **32 GB/s**	1.0 / 15 GB/s

- 单流 H2D 12.9 GB/s（约 PCIe Gen5 x8 的一半，单队列限制）；16 并发聚合 32 GB/s。
- **D2H 明显不对称**（单流 3.4 GB/s），取回大结果时建议多流并发（聚合可到 15–30 GB/s）。
- 小消息（1 MB）往返有效带宽骤降至 ~3 GB/s，launch 开销 ~0.3 ms 量级。

4. 集合通信（nccom-test，bf16，单机 NeuronLink）

每项取 32B 小消息延迟与峰值 busbw：

op	4 ranks (1 chip)	16 ranks (4/chip×4)	64 ranks (整机)
all-reduce	7.4 µs / 195 GB/s	8.2 µs / 199 GB/s	9.9 µs / **325 GB/s**

op	4 ranks (1 chip)	16 ranks (4/chip×4)	64 ranks (整机)
all-gather	5.7 µs / 212 GB/s	11.5 µs / 177 GB/s	22.0 µs / **379 GB/s**
reduce-scatter	8.2 µs / 145 GB/s	11.4 µs / 190 GB/s	14.4 µs / **275 GB/s**
all-to-all	5.7 µs / 91 GB/s	13.3 µs / 26 GB/s	33.8 µs / **22 GB/s**

结论

- 小消息延迟优秀：芯片内 all-reduce 7.4 µs，整机 64 核也只有 ~10 µs，适合 TP 推理的频繁小同步。
- 大消息 all-reduce/all-gather busbw 300+ GB/s（需 ≥512 MB 才饱和，爬坡慢，128 MB 时约为峰值 55%）。
- **all-to-all 是明显短板**：64 ranks 仅 ~22 GB/s busbw，且 8 MiB 就见顶。 专家并行（MoE dispatch/combine）在单机 trn2 上会受此限制，MoE 大模型建议优先 TP/DP 切分而非 EP。

5. 模型基准（vLLM-Neuron 0.29）

配置：`NEURON_SKIP_EFA_AFFINITY=1`， `--no-enable-prefix-caching --max-num-batched-tokens 4096`，输入 1024 / 输出 512 tokens × 64 prompts（吞吐）；输入 128 / 输出 128、batch=1（延迟）。

a) 吞吐（64 并发请求）

模型	TP	场景	req/s	总 tok/s	输出 tok/s
Qwen3-32B	32	1024 in / 128 out（prefill 为主）	5.97	6875	764
Qwen3-32B	32	1024 in / 512 out（decode 为主）	2.15	3297	**1099**
DS-R1-Distill-70B	64	1024 in / 128 out	4.29	4938	549
DS-R1-Distill-70B	64	1024 in / 512 out	1.51	2322	**774**

b) 单流延迟（batch=1，128 in / 128 out）

模型	TP	平均时延	每 token
Qwen3-32B	32	1.66 s	**13.0 ms/tok**（77 tok/s）
DS-R1-Distill-70B	64	1.72 s	**13.4 ms/tok**（74 tok/s）

结论

- 70B 用整机 TP=64 与 32B 用半机 TP=32 的单流延迟几乎持平（13.0 vs 13.4 ms/tok）： 参数量×2.2 被 TP 核数×2 基本抵消，印证 NeuronLink all-reduce 小消息延迟低、TP 扩展效率高。
- decode 阶段输出吞吐 Qwen3-32B@TP32 约 1.1k tok/s、70B@TP64 约 0.77k tok/s（64 并发、bf16、未调优： chunked prefill 限 batch=1、无 continuous batching 调参、无量化）。属 vLLM-Neuron 开箱水平，而非极限。
- 编译成本显著：首次 trace+编译 20–40 min/模型/配置（NVMe 上的 neuron-cache 可复用）。

5.1 DeepSeek MoE 原版兼容性调研（2026-11-08）

结论：**DeepSeek V4 / V4.1-Flash 目前在 Trainium 上无任何可用推理路径；V3/R1 原版可走 NxD Inference 但需另行安排**。

路径	版本	DeepSeek 支持	实测/依据
vLLM-Neuron	0.29.0.1.2.0（已是 PyPI 最新）	❌ 架构白名单仅 `LlamaForCausalLM` / `Qwen3ForCausalLM` / `GptOssForCausalLM`	实测加载 V4-Flash-0731 被拒：`deepseek_v4_fp8 quantization is currently not supported in neuron`
NxD Inference	0.10.18399（最新）	⚠️ 仅 DeepSeek **V3 系**（含 MLA 的 `DeepseekV3Attention`），可跑 V3/R1 671B	包内 `models/deepseek/modeling_deepseek.py`，无任何 V4 代码
其他（社区）	—	❌ 未发现针对 Trainium 的第三方 DeepSeek V4 实现	—

- 本报告用 **R1-Distill-Llama-70B** 代表 DeepSeek，是受支持架构中唯一的官方 DeepSeek 模型 （R1-Distill-Qwen 系为 Qwen2 架构，同样不在白名单）。
- 跑 V3/R1 671B 原版的前置条件：FP8 权重 ~700 GB 需提前下载（本 block 剩余时间不够）， NxDI 路径编译时间 1h 级，建议预留 ≥12h 的 capacity block 并考虑 trn2u/UltraServer。

6. 与 AWS 官方规格核对

官方数据来源：AWS Neuron 文档 Trainium2 架构页、aws.amazon.com/ec2/instance-types/trn2/（2026-11-08 查证）。

指标	本次实测	AWS 官方标称	达成率
BF16 密集算力 / 芯片	594 TF（148.6×4）	667 TFLOPS	**89%**
BF16 密集算力 / 整机	7.59 PF	10.67 PF（667×16）	**71%**
FP32 算力 / 芯片	145 TF（36.3×4）	181 TFLOPS	**80%**
HBM 容量 / 整机	1.5 TB（neuron-ls 16×96GB）	1.5 TB HBM3	✓
HBM 带宽 / 芯片	2.57 TB/s（643×4，DMA r+w）	2.9 TB/s	**89%**
HBM 带宽 / 整机	41.2 TB/s	46 TB/s	**90%**
NeuronLink / 芯片	all-reduce busbw ≈1.30 TB/s（325×4）	1.28 TB/s (v3)	**≈100%**
FP8 算力	未测（框架 bf16 路径）	1299 TFLOPS/芯片	—
EFA 网络	本实例未挂载	3.2 Tbps EFAv3	—

核对结论

- **HBM 带宽（89–90%）与 NeuronLink（≈100%）实测与官方规格高度吻合**，是最硬的两个验证点；其中 64-rank all-reduce 的 per-chip 总线带宽恰好打满 NeuronLink-v3 标称值，说明集合通信实现已达链路极限。
- GEMM 单芯片 89% 达成率属正常编译器/微基准水平（官方峰值为脉动阵列理论值）； 整机 71% 的缺口来自芯片内 4 逻辑核并发时共享 HBM/功耗（见 §1），与官方数据不矛盾。

- 单核 STREAM/elementwise 偏低（§2b）是计算引擎路径特性，与 HBM 规格无关，DMA 路径已验证 89%。
- 官方另标注每芯片 DMA 3.5 TB/s、SBUF 224 MiB、8×NeuronCore-v3 + 16 CC-Cores，与 neuron-ls/ 实测行为一致。

7. 踩坑记录（可复现性）

1. **多进程必须用 `NEURON_RT_VISIBLE_CORES` 按 rank 分核**（在 import torch_neuronx 之前设置），否则 rank0 独占全部 64 逻辑核，其余 rank 报 `cores busy, ret=-16`。
2. 无 EFA 的 trn2 上 vLLM-Neuron 需 `NEURON_SKIP_EFA_AFFINITY=1`，否则 EngineCore 启动失败。
3. vLLM-Neuron 默认开 APC 但要求 segmented prefill：`--no-enable-prefix-caching` 或 `--max-num-batched-tokens ∈ {512..8192}`。
4. 单核超大 DMA copy（4 GB×2）会撞 NRT 30 s 执行超时（FATAL-RT-UNDEFINED-STATE），分块即可。
5. nccom-test 的 op 名是 `allr/allg/redsct/alltoall`，无 `--check none` 选项。

附录

- 原始数据：`results/*.jsonl`、`results/collectives/*.txt`、`results/summary_tables.md`
- 复现：`scripts/00_setup.sh` → `run_micro.sh` / `finish_suite.sh` → `06_model_bench.sh`